Freshman Tutorial Platform

Design and Implementation Document · Natural Science and Social Science Freshman Courses plus a Web Development Skills Track

Stack: HTML, CSS, JavaScript, PHP · Frontend: Netlify · Backend: Render · Database: MongoDB Atlas

1. Project Overview

The Freshman Tutorial Platform is a web-based learning system for first-year university students. It covers the **common freshman courses of the Natural Science and Social Science streams**, plus a **Web Development skills track** that students can take alongside them (it also supports the Emerging Technologies course). Students read lessons, practice with exercises, take quizzes, join course discussions and track their progress. Instructors and tutors create and manage the content.

The system has three parts: a static frontend, a PHP REST API and a cloud database.

2. Problem Statement

Freshman students study many different courses at once, and the learning material is scattered across textbooks, slides, social media groups and websites. It is rarely organized by stream or course, and students cannot practice or measure their progress in one place. Instructors have no central place to publish lessons, collect questions and see how students are doing. Students who want to learn web development as an extra skill have no beginner path either.

3. Objectives

- Provide structured tutorials for **all freshman courses of both streams**, each with lessons, exercises and quizzes.
- Provide a beginner-to-intermediate **Web Development track** (HTML, CSS, JavaScript, PHP) with a live code playground.
- Let each student choose a stream and automatically see the right course list.
- Track each student's progress and show it on a dashboard.

- Let instructors, tutors and admins create, update and manage courses, lessons, quizzes and announcements.
- Deploy on free cloud services so the platform is reachable from any device, including low-end phones.

4. Freshman Course Catalog

The course lists below were merged and cleaned. Courses shared by both streams are stored **once** in the database and tagged with every stream that uses them, so content is never duplicated.

4.1 Natural Science stream

No.	Course
1	General Physics
2	Mathematics for Natural Sciences
3	Communicative English I
4	Social Anthropology
5	Geography
6	Emerging Technologies
7	Moral and Citizenship Education
8	Global Trends
9	General Psychology
10	Logic and Critical Thinking
11	Sport (Physical Fitness)

4.2 Social Science stream

No.	Course
1	English Skills I (Communicative English I)

No.	Course
2	Economics
3	Global Trends
4	Social Anthropology
5	Emerging Technologies
6	Entrepreneurship (listed twice in the source, merged into one course)
7	History
8	Inclusiveness
9	General Psychology
10	Logic and Critical Thinking
11	Sport (Physical Fitness)

4.3 Merged catalog (single source of truth)

Course	Natural	Social	Group
English (Communicative English I / English Skills I)	✔	✔	Common
Social Anthropology	✔	✔	Common
Emerging Technologies	✔	✔	Common
Global Trends	✔	✔	Common
General Psychology	✔	✔	Common
Logic and Critical Thinking	✔	✔	Common
Sport (Physical Fitness)	✔	✔	Common
General Physics	✔		Natural only
Mathematics for Natural Sciences	✔		Natural only
Geography	✔		Natural only

Course	Natural	Social	Group
Moral and Citizenship Education	✔		Natural only
Economics		✔	Social only
Entrepreneurship		✔	Social only
History		✔	Social only
Inclusiveness		✔	Social only
Web Development (HTML, CSS, JavaScript, PHP)	optional	optional	Digital skills track

Result: 7 common courses, 4 Natural-only courses, 4 Social-only courses and 1 optional skills track, 16 course shells in total instead of 22 duplicated entries.

> Note: Moral and Citizenship Education appears only in the Natural list you supplied. If your university offers it to both streams, add `"social"` to its `streams` array in the seed data (section 11.1); no code changes are needed.

5. User Roles

Role	Responsibilities
Admin	Manages users, roles, streams, courses, announcements, system settings and reports
Instructor	Creates courses, lessons, exercises and quizzes; reviews student progress; answers questions
Tutor / Teaching Assistant	Helps students, answers forum questions, reviews submitted exercises
Student (Freshman)	Registers, picks a stream, enrolls, learns, practices, takes quizzes, tracks progress, earns certificates
Guest	Views the course list and free sample lessons before registering

6. System Modules

#	Module	Main features
1	Authentication and Authorization	Register, login, logout, password reset, role-based access, JWT tokens
2	User Profile	Edit profile, change password, avatar, **choose stream (Natural or Social)**, learning history
3	Course Management	Create, edit, delete and publish courses; assign streams, groups and levels
4	Lesson Management	Lessons with text, images, formulas, code samples and video links; ordering; downloadable notes
5	Code Playground	Live HTML, CSS and JavaScript editor with instant preview in a sandboxed iframe (used by Web Dev and Emerging Technologies)
6	Exercise and Assignment	Practice tasks, student submission, instructor feedback and grading
7	Quiz and Assessment	Multiple choice and true/false quizzes, auto grading, score history, per-course question banks
8	Progress Tracking	Completed lessons, quiz scores, percentage per course, dashboard charts
9	Discussion Forum	Students ask questions per course; tutors and instructors answer
10	Search and Filter	Search by keyword, stream, group, course and level
11	Announcements and Notifications	Admin and instructor announcements, in-app notifications, exam reminders
12	Certificates	Auto-generated printable certificate on course completion
13	Reports and Analytics	Enrollment, completion rate, quiz performance, active users per stream
14	Admin Panel	Manage users, content, streams and system data

#	Module	Main features
15	Study Tools (new)	Formula sheets for Physics and Mathematics, flashcards for memorization-heavy courses, Sport fitness log

7. Learning Content Plan

Every course follows the same unit pattern so the platform can render it with one template: **Unit → Lessons → Exercise → Quiz**. Suggested content focus per course:

Course	Content and tool support
General Physics	Mechanics, heat, waves, electricity; formula sheet, worked examples, numeric quizzes
Mathematics for Natural Sciences	Functions, limits, derivatives, integrals; math rendering with KaTeX, step-by-step solutions
English (Communicative English I / English Skills I)	Reading, writing, grammar, listening and speaking practice; vocabulary flashcards
Social Anthropology	Culture, society, kinship, social institutions; case studies, short-answer reflection
Geography	Earth systems, climate, population, maps and images
Emerging Technologies	AI, IoT, data science, cloud, cybersecurity basics; links to the Web Development track
Moral and Citizenship Education	Ethics, rights and duties, democracy; scenario quizzes
Global Trends	Globalization, international relations, global issues; timeline and news-style lessons
General Psychology	Learning, memory, motivation, personality; concept flashcards
Logic and Critical Thinking	Arguments, fallacies, reasoning; interactive true/false and puzzle quizzes

Course	Content and tool support
Sport (Physical Fitness)	Fitness principles, exercises, nutrition and health; video links and a personal activity log
Economics	Demand and supply, markets, national income; charts and calculation quizzes
Entrepreneurship	Idea generation, business plan, marketing, finance basics; mini business-plan exercise
History	Ethiopia and the Horn, major periods and events; timelines and map images
Inclusiveness	Disability, gender, diversity and inclusion; scenario discussions
Web Development (skills track)	1. HTML basics · 2. CSS basics (Flexbox, Grid, responsive) · 3. JavaScript basics (DOM, events, fetch) · 4. PHP basics (forms, sessions, REST API, MongoDB) · 5. Mini project: personal website or to-do app

8. System Architecture

```
Student / Instructor / Admin (Browser, Mobile)
        |
        | HTTPS
        v
+--------------------------------+
| FRONTEND (Netlify)             |
| HTML + CSS + JavaScript        |
| Static pages, calls the API    |
+--------------------------------+
        |
        | fetch() / JSON over HTTPS (CORS restricted)
        v
+--------------------------------+
| BACKEND API (Render)           |
| PHP REST API (Docker)          |
| Auth, roles, business logic    |
+--------------------------------+
```

```
        |
        | MongoDB driver (TLS)
        v
+------------------------------+
| DATABASE (MongoDB Atlas, M0)   |
| users, courses, lessons, ...   |
+------------------------------+
```

Important: Netlify serves static files only, so no PHP runs there. All PHP lives on Render and the frontend talks to it through JSON calls.

9. Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (ES6)	Pages, styling, interaction, API calls
UI helpers	Bootstrap or Tailwind, Chart.js	Responsive layout, progress charts
Math display	KaTeX	Physics and Mathematics formulas
Code editor	CodeMirror or Monaco (optional)	Playground editor
Backend	PHP 8.2, Apache (Docker)	REST API
PHP libraries	mongodb/mongodb, firebase/php-jwt, vlucas/phpdotenv	Database, JWT, environment variables
Database	MongoDB Atlas free M0 cluster	Data storage
Hosting	Netlify (frontend), Render (backend)	Deployment
Tools	VS Code, Git, GitHub, Postman	Development, version control, API testing

10. Database Design (MongoDB)

users

```
{ "_id": ObjectId, "fullName": "", "email": "", "passwordHash": "",
  "role": "student|instructor|tutor|admin",
  "stream": "natural|social|null",
  "avatarUrl": "", "isActive": true, "createdAt": Date }
```

courses (changed: streams, group, code)

```
{ "_id": ObjectId, "code": "PHYS101", "title": "General Physics",
  "description": "",
  "streams": ["natural"],
  "group": "common|natural|social|digital",
  "level": "beginner|intermediate",
  "instructorId": ObjectId, "isPublished": true, "createdAt": Date }
```

lessons

```
{ "_id": ObjectId, "courseId": ObjectId, "unit": 1, "title": "",
  "content": "", "codeExample": "", "videoUrl": "",
  "order": 1, "isFree": false }
```

enrollments

```
{ "_id": ObjectId, "studentId": ObjectId, "courseId": ObjectId,
  "enrolledAt": Date, "completedLessons": [ObjectId],
  "progressPercent": 0, "completedAt": null }
```

quizzes and quizAttempts

```
{ "_id": ObjectId, "courseId": ObjectId, "lessonId": ObjectId, "title": "",
  "questions": [ { "text": "", "options": [""], "correctIndex": 0 } ] }

{ "_id": ObjectId, "quizId": ObjectId, "studentId": ObjectId,
  "score": 0, "total": 0, "answers": [], "takenAt": Date }
```

exercises and submissions

```
{ "_id": ObjectId, "lessonId": ObjectId, "instruction": "", "starterCode": ""
}

{ "_id": ObjectId, "exerciseId": ObjectId, "studentId": ObjectId,
  "code": "", "status": "pending|reviewed", "grade": 0,
  "feedback": "", "submittedAt": Date }
```

forumPosts

```
{ "_id": ObjectId, "courseId": ObjectId, "authorId": ObjectId, "title": "",
  "body": "",
  "replies": [ { "authorId": ObjectId, "body": "", "createdAt": Date } ],
  "createdAt": Date }
```

`announcements`, `notifications`, `certificates`, `fitnessLogs` and `auditLogs` follow the same pattern (id, owner or target, message or data, createdAt).

Recommended indexes: `users.email` (unique), `courses.streams`, `lessons.courseId + order`, `enrollments.studentId + courseId` (unique), `quizAttempts.studentId`.

11. Implementation

11.1 Seed data: loading the course catalog

Create `backend/seed/courses.json` and load it once with a small PHP script. Shared courses carry both streams, which removes duplicates.

```
[
  { "code": "ENGL101", "title": "Communicative English I", "streams":
["natural","social"], "group": "common" },
  { "code": "ANTH101", "title": "Social Anthropology", "streams":
["natural","social"], "group": "common" },
  { "code": "EMTE101", "title": "Emerging Technologies", "streams":
["natural","social"], "group": "common" },
  { "code": "GLTR101", "title": "Global Trends", "streams":
["natural","social"], "group": "common" },
  { "code": "PSYC101", "title": "General Psychology", "streams":
["natural","social"], "group": "common" },
  { "code": "LOCT101", "title": "Logic and Critical Thinking", "streams":
["natural","social"], "group": "common" },
  { "code": "SPRT101", "title": "Sport (Physical Fitness)", "streams":
["natural","social"], "group": "common" },
  { "code": "PHYS101", "title": "General Physics", "streams": ["natural"],
"group": "natural" },
  { "code": "MATH101", "title": "Mathematics for Natural Sciences",
"streams": ["natural"], "group": "natural" },
  { "code": "GEOG101", "title": "Geography", "streams": ["natural"], "group":
"natural" },
  { "code": "MCEN101", "title": "Moral and Citizenship Education", "streams":
["natural"], "group": "natural" },
  { "code": "ECON101", "title": "Economics", "streams": ["social"], "group":
"social" },
  { "code": "ENTR101", "title": "Entrepreneurship", "streams": ["social"],
"group": "social" },
  { "code": "HIST101", "title": "History", "streams": ["social"], "group":
"social" },
  { "code": "INCL101", "title": "Inclusiveness", "streams": ["social"],
"group": "social" },
  { "code": "WEBD101", "title": "Web Development", "streams":
["natural","social"], "group": "digital" }
]
```

Course codes above are suggestions; replace them with your university’s official codes.

11.2 Router entry point (`public/index.php`)

```
<?php
require __DIR__ . '/../vendor/autoload.php';
Dotenv\Dotenv::createImmutable(__DIR__ . '/..')->safeLoad();

Cors::handle();                       // sets headers, answers OPTIONS
$method = $_SERVER['REQUEST_METHOD'];
$path   = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);

$routes = [
  ['POST', '#^/api/auth/register$#',    [AuthController::class, 'register'],
null],
  ['POST', '#^/api/auth/login$#',       [AuthController::class, 'login'],
null],
  ['GET',  '#^/api/auth/me$#',          [AuthController::class, 'me'],
['any']],
  ['GET',  '#^/api/courses$#',          [CourseController::class, 'list'],
null],
  ['POST', '#^/api/courses$#',          [CourseController::class, 'create'],
['instructor','admin']],
  ['POST', '#^/api/enrollments$#',      [EnrollmentController::class,
'create'], ['student']],
];

foreach ($routes as [$m, $pattern, $handler, $roles]) {
  if ($m === $method && preg_match($pattern, $path, $args)) {
    $user = $roles ? AuthMiddleware::require($roles) : null;  // 401 / 403
handled inside
    (new $handler[0])->{$handler[1]}($user, array_slice($args, 1));
    exit;
  }
}
Response::json(false, null, 'Not found', 404);
```

11.3 Stream-aware course listing

```
public function list($user, $args) {
  $filter = ['isPublished' => true];
  $stream = $_GET['stream'] ?? null;
  if (in_array($stream, ['natural', 'social'], true)) {   // whitelist, no
raw input in the query
    $filter['streams'] = $stream;                          // matches arrays
containing the value
  }
  $group = $_GET['group'] ?? null;
  if (in_array($group, ['common','natural','social','digital'], true)) {
    $filter['group'] = $group;
  }
  $courses = $this->db->courses->find($filter, ['sort' => ['title' => 1]])-
>toArray();
  Response::json(true, $courses, 'OK');
}
```

11.4 Auth middleware (JWT and roles)

```
use Firebase\JWT\JWT;
use Firebase\JWT\Key;

class AuthMiddleware {
  public static function require(array $roles) {
    $header = $_SERVER['HTTP_AUTHORIZATION'] ?? '';
    if (!preg_match('/^Bearer\s+(.+)$/', $header, $m)) Response::json(false,
null, 'Unauthorized', 401);
    try {
      $payload = JWT::decode($m[1], new Key($_ENV['JWT_SECRET'], 'HS256'));
    } catch (Throwable $e) {
      Response::json(false, null, 'Invalid or expired token', 401);
    }
    if (!in_array('any', $roles, true) && !in_array($payload->role, $roles,
true)) {
      Response::json(false, null, 'Forbidden', 403);
    }
    return $payload;   // contains sub (user id), role, stream, exp
  }
}
```

11.5 CORS helper (supports exact origin match)

```
class Cors {
  public static function handle() {
    $allowed = array_map('trim', explode(',', getenv('FRONTEND_URL')));
    $origin  = $_SERVER['HTTP_ORIGIN'] ?? '';
    if (in_array($origin, $allowed, true)) {
      header('Access-Control-Allow-Origin: ' . $origin);
      header('Vary: Origin');
    }
    header('Access-Control-Allow-Headers: Content-Type, Authorization');
    header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
    if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') { http_response_code(204);
exit; }
  }
}
```

`FRONTEND_URL` may hold several comma-separated origins, for example the Netlify URL and `http://localhost:5500` during development.

11.6 Frontend API helper with sleep-wake message (`js/api.js`)

```
async function api(path, method = "GET", body) {
  const token = localStorage.getItem("token");
  const banner = document.getElementById("loading");
  const slowTimer = setTimeout(() => banner && (banner.hidden = false),
3000);
  try {
    const res = await fetch(API_BASE_URL + path, {
      method,
      headers: {
        "Content-Type": "application/json",
        ...(token && { Authorization: "Bearer " + token })
      },
      body: body ? JSON.stringify(body) : undefined
    });
    const json = await res.json();
    if (res.status === 401) { localStorage.removeItem("token"); location.href
= "login.html"; }
    return json;
  } finally {
```

```
    clearTimeout(slowTimer);
    if (banner) banner.hidden = true;
  }
}
```

The banner text can read: “Waking up the server, this can take up to a minute on the first visit.”

11.7 Stream-based home page (`js/courses.js`)

```
async function loadMyCourses() {
  const me = await api("/auth/me");
  const stream = me.data.stream;                    // "natural" or "social"
  const res = await api(`/courses?stream=${stream}`);
  const box = document.getElementById("courses");
  box.textContent = "";
  res.data.forEach(c => {
    const card = document.createElement("a");
    card.className = "course-card";
    card.href = `lesson.html?course=${c._id.$oid}`;
    card.textContent = c.title;                     // textContent, never
innerHTML (XSS safe)
    box.appendChild(card);
  });
}
```

11.8 Progress calculation

When a student completes a lesson, the API adds the lesson id with `$addToSet` (so repeats are ignored), then recalculates:

```
progressPercent = round( completedLessons.length / totalLessonsInCourse * 100
)
```

When the value reaches 100 and the final quiz is passed (score of at least 50%), set `completedAt` and create a certificate record.

12. REST API Endpoints

All responses use one shape: `{ "success": true, "data": ..., "message": "" }`.

Area	Method and path	Access
Auth	POST /api/auth/register	Public
Auth	POST /api/auth/login	Public
Auth	GET /api/auth/me	Logged in
Profile	PUT /api/users/me (name, avatar, stream)	Logged in
Courses	GET /api/courses?stream=&group=&q=	Public
Courses	POST /api/courses	Instructor, Admin
Courses	PUT and DELETE /api/courses/{id}	Owner, Admin
Lessons	GET /api/courses/{id}/lessons	Enrolled, or public if free
Lessons	POST /api/lessons · PUT /api/lessons/{id}	Instructor, Admin
Enroll	POST /api/enrollments	Student
Progress	POST /api/progress/complete-lesson	Student
Progress	GET /api/progress/me	Student
Quiz	GET /api/quizzes/{id} · POST /api/quizzes/{id}/attempt	Student
Exercises	POST /api/submissions · PUT /api/submissions/{id}/grade	Student / Instructor
Forum	GET and POST /api/forum · POST /api/forum/{id}/reply	Logged in
Announcements	GET /api/announcements · POST /api/announcements	Logged in / Instructor, Admin
Admin	GET /api/admin/users · PUT /api/admin/users/{id}/role	Admin
Reports	GET /api/reports/summary	Admin, Instructor

13. Folder Structure

```
freshman-tutorial/
├── frontend/                 (deployed to Netlify)
|   ├── index.html            landing page, stream picker
|   ├── login.html
|   ├── register.html
|   ├── courses.html          filter by stream, group, keyword
|   ├── lesson.html           lesson viewer (text, formulas, video)
|   ├── playground.html       HTML/CSS/JS live editor
|   ├── quiz.html
|   ├── dashboard.html        progress charts per course
|   ├── forum.html
|   ├── certificate.html      printable certificate
|   ├── admin/
|   |   ├── users.html
|   |   ├── courses.html
|   |   └── reports.html
|   ├── css/style.css
|   └── js/
|       ├── config.js         API_BASE_URL
|       ├── api.js            fetch helper and token
|       ├── auth.js
|       ├── courses.js
|       ├── playground.js
|       └── dashboard.js
|
└── backend/                  (deployed to Render)
    ├── Dockerfile
    ├── composer.json
    ├── seed/courses.json     course catalog (section 11.1)
    ├── seed/load.php         one-time loader
    ├── public/
    |   ├── index.php         router entry point
    |   └── .htaccess
    ├── src/
    |   ├── config/Database.php
    |   ├── controllers/      Auth, Course, Lesson, Enrollment, Quiz, Forum,
...
```

```
    |   ├── middleware/       AuthMiddleware, Cors
    |   ├── models/
    |   └── helpers/Response.php
    └── .env.example
```

The `.htaccess` file must send every request to `index.php`:

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteRule ^ index.php [QSA,L]
```

14. Security Design

- Hash passwords with `password_hash()` (bcrypt) and check them with `password_verify()`.
- Issue JWTs with an expiry (for example 24 hours) and send them in the `Authorization: Bearer` header.
- Check the role on **every** protected endpoint on the server; never trust the frontend alone.
- Validate and sanitize all input, reject unexpected fields, and whitelist filter values such as `stream` and `group`.
- Never insert user text with `innerHTML`; use `textContent` or an HTML sanitizer, because lesson and forum content is user-written.
- Because the token is kept in `localStorage`, any XSS bug could steal it. Keep tokens short-lived and add a strict Content-Security-Policy header in Netlify's `_headers` file.
- Use typed values in MongoDB filters to prevent NoSQL injection; cast ids with `new MongoDB\BSON\ObjectId()` inside try/catch.
- Limit CORS to the Netlify domain only.
- Rate-limit login attempts (for example 5 failures per 15 minutes per email and IP).
- Keep secrets (database URI, JWT secret) in Render environment variables, never in Git.
- Run the code playground in `<iframe sandbox="allow-scripts">`; never execute student PHP on the server.

15. Deployment Guide

Step 1: MongoDB Atlas

1. Create a free account and an M0 cluster.
2. Create a database user with a strong password and read/write access to `tutorialdb` only.
3. Under Network Access, allow Render. The free plan has no fixed IP, so `0.0.0.0/0` is usually needed; compensate with strong credentials and a limited user.
4. Copy the connection string, for example `mongodb+srv://USER:PASS@cluster.mongodb.net/tutorialdb`.
5. Run `seed/load.php` once (locally or via a Render shell) to insert the course catalog and create the indexes.

Step 2: Backend on Render

Render has no native PHP runtime, so deploy with Docker.

```
FROM php:8.2-apache
RUN apt-get update && apt-get install -y git unzip libssl-dev pkg-config \
 && pecl install mongodb \
 && docker-php-ext-enable mongodb \
 && a2enmod rewrite headers
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
WORKDIR /var/www/html
COPY . .
RUN composer install --no-dev --optimize-autoloader
ENV APACHE_DOCUMENT_ROOT /var/www/html/public
RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g'
/etc/apache2/sites-available/*.conf \
 && sed -ri -e 's!/var/www/!${APACHE_DOCUMENT_ROOT}!g'
/etc/apache2/apache2.conf /etc/apache2/conf-available/*.conf
EXPOSE 80
```

1. Push the project to GitHub.
2. On Render choose **New → Web Service**, connect the repository, set the root directory to `backend` and the runtime to **Docker**.
3. Add environment variables: `MONGODB_URI`, `DB_NAME`, `JWT_SECRET`, `FRONTEND_URL`.
4. Deploy and test with Postman: `https://your-api.onrender.com/api/courses`.

> The Render free plan sleeps after inactivity, so the first request can take 30 to 60 seconds. The loading banner in section 11.6 handles this, and a free uptime pinger can keep the service awake.

Step 3: Frontend on Netlify

1. Choose **Add new site → Import from Git** and select the repository.
2. Set the base and publish directory to `frontend` (no build command is needed).
3. In `js/config.js` set `const API_BASE_URL = "https://your-api.onrender.com/api";`.
4. Deploy; Netlify gives a URL such as `https://freshman-tutorial.netlify.app`.
5. Put that URL in the Render variable `FRONTEND_URL` so CORS allows it.

Environment variables

Variable	Where	Example
MONGODB_URI	Render	mongodb+srv://user:pass@cluster.mongodb.net
DB_NAME	Render	tutorialdb
JWT_SECRET	Render	long random string (at least 32 characters)
FRONTEND_URL	Render	https://freshman-tutorial.netlify.app
API_BASE_URL	Frontend config.js	https://your-api.onrender.com/api

16. Development Plan (10 Weeks)

Week	Tasks
1	Requirements, course catalog confirmation with the department, UI sketches, database design, Git repository
2	Atlas setup, PHP skeleton, router, database connection, seed script

Week	Tasks
3	Authentication, roles, JWT, stream selection, login and register pages
4	Course and lesson management (API and admin pages)
5	Student pages: stream-based course list, lesson viewer with formulas, enrollment
6	Code playground, exercises, flashcards
7	Quizzes and progress dashboard
8	Forum, announcements, certificates, Sport fitness log
9	Deployment to Render and Netlify, testing, bug fixing
10	Content entry for priority courses, documentation, final report, presentation

Content tip: Do not wait until the end to write lessons. Start with one common course (for example Logic and Critical Thinking), one Natural course (Mathematics) and one Social course (Economics) so every feature is tested with real material by week 7.

17. Testing Plan

- **API testing:** a Postman collection covering every endpoint.
- **Functional testing:** register, choose stream, enroll, complete lesson, take quiz, submit exercise.
- **Stream testing:** a Natural student never sees Social-only courses in the default list, and shared courses appear once for both.
- **Role testing:** each role reaches only its allowed pages and endpoints.
- **Responsive testing:** desktop, tablet and phone, including slow connections.
- **Security testing:** invalid tokens, wrong roles, malformed ids, script injection in forum posts.
- **Usability testing:** a few real freshman students from both streams try the platform and give feedback.

18. Risks and Limitations

Risk	Mitigation
Render free plan sleeps (slow first request)	Loading banner; optional free uptime pinger
Atlas free tier has a 512 MB limit	Keep videos as external links (YouTube); store only text and metadata; compress images
Netlify cannot run PHP	Keep all PHP in the Render backend
Public database access (0.0.0.0/0)	Strong credentials, least-privilege user, secrets in environment variables
Students running unsafe code	Sandboxed iframe; no server-side execution of student code
Large amount of content for 16 courses	Common template per course, start with priority courses, let instructors contribute in parallel
Copyright of textbooks and slides	Publish only original or permitted material; link to official sources
Limited data or internet for students	Light pages, no heavy libraries, optional PWA offline mode later

19. Expected Results

A responsive website where Natural Science and Social Science freshmen find all their common courses in one place, learn through lessons, practice and quizzes, and track their progress, while also learning HTML, CSS, JavaScript and PHP in an optional skills track. Instructors and administrators manage content and monitor performance, all on free cloud services.

20. Future Enhancements

- Multilingual support (Amharic, Afan Oromo and others)
- Automatic code checking for exercises
- Video hosting and downloadable resources

- Progressive Web App with offline lessons
- Badges, points and leaderboards
- AI-based learning assistant for course questions
- Email and SMS notifications
- Exit-exam style practice banks

21. Suggested Project Titles

1. Design and Implementation of a Web-Based Interactive Tutorial Platform for Natural and Social Science Freshman Courses with Progress Tracking and a Web Development Code Playground
2. Development of a Cloud-Deployed E-Learning System for Freshman Students Using HTML, CSS, JavaScript, PHP and MongoDB Atlas